

1. 使用Anaconda创建环境

```
# 创建环境
conda create -n llm_learn python=3.11.10
```

1.1 为jupyter notebook添加所有环境(如有需要)

```
# 激活环境
conda activate llm_learn

# 安装ipykernel
conda install ipykernel
#退出环境
conda deactivate

#激活默认环境
conda activate base
#安装 nb_conda_kernels
conda install nb_conda_kernels
```

2. 数据准备练习—使用爬虫爬取图片

2.1 安装所需要的库

(仅为简单案例所需，想要了解更高级爬虫算法可自行研究)

```
pip install requests #清华源：https://pypi.tuna.tsinghua.edu.cn/simple
pip install beautifulsoup4
```

2.2 从Bing图片库中爬取雪天图像

关键词（search_term）可替换为其他元素，此处使用的关键词为“**snow street real-world person auto-driven**”

网址 (url) 也可以更换为其他图片网站

```
def download_snow_images(num_images=50, save_dir='snow_images'):
    # 创建保存图片的目录
    if not os.path.exists(save_dir): os.makedirs(save_dir)
    # Bing图片搜索的URL
    search_term = 'snow street real-world person auto-driven'
    url = f'https://www.bing.com/images/search?q={urllib.parse.quote(search_term)}\&first=1&tsc=ImageBasicHover'

    # 发送GET请求
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')

    # 查找所有图片链接
    img_tags = soup.find_all('img', class_='mimg')

    # 下载图片
    count = 0
    for img in img_tags:
        if count >= num_images:
            break

        img_url = img.get('src')
        if img_url and img_url.startswith('http'):
            try:
                img_data = requests.get(img_url).content
                with open(os.path.join(save_dir, f'snow_image_{count}.jpg'), 'wb') as handler:
                    handler.write(img_data)
                print(f'Downloaded image {count + 1}')
                count += 1
            except Exception as e:
                print(f'Error downloading image: {e}')
    print(f'Downloaded {count} images to {save_dir}')

# 调用函数下载50张雪天图片
download_snow_images(10)
```

结果：



snow_image_0.j
pg



snow_image_1.j
pg



snow_image_2.j
pg



snow_image_3.j
pg



snow_image_4.j
pg



snow_image_5.j
pg



snow_image_6.j
pg



snow_image_7.j
pg



snow_image_8.j
pg

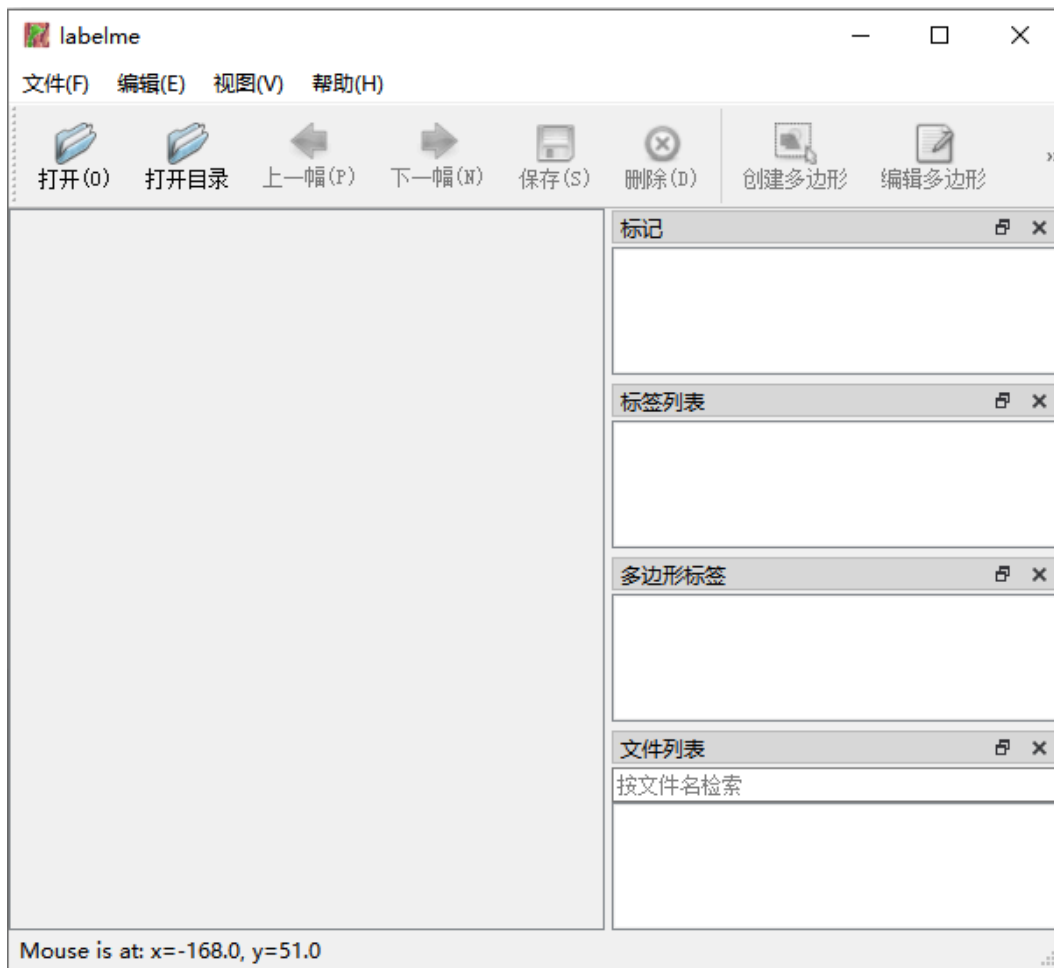


snow_image_9.j
pg

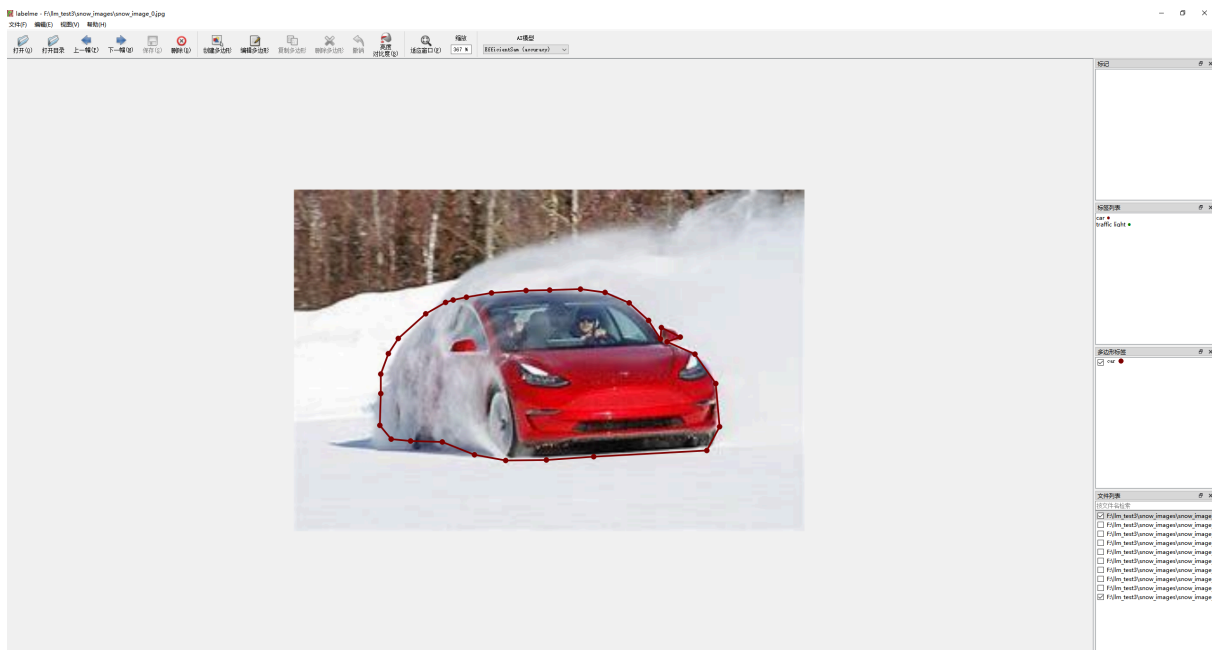
2.3 使用labelMe对图像进行标注

2.3.1 安装labelMe，启动labelme

```
# 安装依赖包  
pip install labelme  
# 启动labelme  
labelme
```



2.3.2 打开待标注文件夹 (f:\snow_images) —> 标注包围框 Bounding Box、掩码 Mask



3. 模型训练练习——训练一个简单的数字图片分类器

3.1 安装所需库

```
# 安装CPU版本的pytorch
pip install torch==2.5.0 torchvision==0.20.0
torchaudio==2.5.0 --index-url https://download.pytorch.org/whl/cpu
```

3.2 定义一个简单的分类网络

可学习层包含两层卷积、两层全连接层

不可学习层为两层池化以及激活函数

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 20, 5, 1)
        self.conv2 = nn.Conv2d(20, 50, 5, 1)
        self.fc1 = nn.Linear(4*4*50, 500)
        self.fc2 = nn.Linear(500, 10)
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2, 2)
        x = x.view(-1, 4*4*50)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.log_softmax(x, dim=1)
```

3.3 加载数据集

此处选择经典的MNIST数据集

在安装PyTorch时，也安装了torchvision，其包含MINST数据集

```
train_loader = torch.utils.data.DataLoader(
    datasets.MNIST('data', train=True, download=True,
                   transform=transforms.Compose([
                       transforms.ToTensor(),
                       transforms.Normalize((0.1307,), (0.3081,))])),
    batch_size=64, shuffle=True)
'''
datasets :
    'data': 数据存放的路径
    train : True下载训练集；false下载测试集
    download : True为下载数据，False为加载本地数据
    transform : 数据变换策略，即数据增强
'''
```

自定义的Dataset类的基本结构：

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
```

3.4 初始化模型和优化器

将模型加载到CPU上

使用SGD优化器，设置学习率，权重衰减，动量更新

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net().to(device)
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

3.5 训练模型

训练3个epoch，batchsize设置为64

```
def train(model, device, train_loader, optimizer, epoch):
    # 模型调整为训练模式，启用 batch normalization 和 dropout
    model.train()
    # 分batch从数据集中取数据
    for batch_idx, (data, target) in enumerate(train_loader):
        # 解析一个batch中的数据
        data, target = data.to(device), target.to(device)
        # 梯度清零
        optimizer.zero_grad()
        # 数据输入模型
        output = model(data)
        # 计算损失函数
        loss = F.nll_loss(output, target)
        # 反向传播
        loss.backward()
        # 参数更新
        optimizer.step()
        # 打印信息
        if batch_idx % 100 == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(epoch,
batch_idx * len(data), len(train_loader.dataset),
100. * batch_idx / len(train_loader), loss.item()))
```

```
for epoch in range(1, 4):
    train(model, device, train_loader, optimizer, epoch)
```

训练过程展示：

```
Train Epoch: 1 [0/60000 (0%)] Loss: 2.352561
Train Epoch: 1 [6400/60000 (11%)] Loss: 0.262660
```

```
Train Epoch: 1 [12800/60000 (21%)] Loss: 0.279559
Train Epoch: 1 [19200/60000 (32%)] Loss: 0.118793
Train Epoch: 1 [25600/60000 (43%)] Loss: 0.071600
Train Epoch: 1 [32000/60000 (53%)] Loss: 0.229571
Train Epoch: 1 [38400/60000 (64%)] Loss: 0.250738
Train Epoch: 1 [44800/60000 (75%)] Loss: 0.143696
Train Epoch: 1 [51200/60000 (85%)] Loss: 0.101742
Train Epoch: 1 [57600/60000 (96%)] Loss: 0.205126
Train Epoch: 2 [0/60000 (0%)] Loss: 0.103794
Train Epoch: 2 [6400/60000 (11%)] Loss: 0.065250
Train Epoch: 2 [12800/60000 (21%)] Loss: 0.042727
Train Epoch: 2 [19200/60000 (32%)] Loss: 0.168471
Train Epoch: 2 [25600/60000 (43%)] Loss: 0.288069
Train Epoch: 2 [32000/60000 (53%)] Loss: 0.068596
Train Epoch: 2 [38400/60000 (64%)] Loss: 0.128328
Train Epoch: 2 [44800/60000 (75%)] Loss: 0.058840
Train Epoch: 2 [51200/60000 (85%)] Loss: 0.097784
Train Epoch: 2 [57600/60000 (96%)] Loss: 0.097051
Train Epoch: 3 [0/60000 (0%)] Loss: 0.031310
Train Epoch: 3 [6400/60000 (11%)] Loss: 0.044818
Train Epoch: 3 [12800/60000 (21%)] Loss: 0.044078
Train Epoch: 3 [19200/60000 (32%)] Loss: 0.044144
Train Epoch: 3 [25600/60000 (43%)] Loss: 0.068360
Train Epoch: 3 [32000/60000 (53%)] Loss: 0.106534
Train Epoch: 3 [38400/60000 (64%)] Loss: 0.011405
Train Epoch: 3 [44800/60000 (75%)] Loss: 0.015950
Train Epoch: 3 [51200/60000 (85%)] Loss: 0.114562
Train Epoch: 3 [57600/60000 (96%)] Loss: 0.054172
```

3.6 测试模型

加载测试集

计算模型预测的准确率

加载测试数据

```
test_loader = torch.utils.data.DataLoader(
    datasets.MNIST('data', train=False, transform=transforms.Compose([
        transforms.ToTensor(),
```



```

        transforms.Normalize((0.1307,), (0.3081,))
    ])),
    batch_size=1000, shuffle=True)

# 测试代码
def test(model, device, test_loader):
    # 切换到验证模式，关闭 batch normalization 和 dropout
    model.eval()
    # 指标数值初始化
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            # 计算负对数似然损失
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            # 获得类别预测
            pred = output.argmax(dim=1, keepdim=True)
            # 计算准确率
            correct += pred.eq(target.view_as(pred)).sum().item()
    test_loss /= len(test_loader.dataset)
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{}
    ({:.0f}%) \n'.format(test_loss, correct, len(test_loader.dataset), 100. * correct /
    len(test_loader.dataset)))

```

结果：

```
Test set: Average loss: 0.0473, Accuracy: 9857/10000 (99%)
```

压缩前模型参数量：

```

total = sum([param.nelement() for param in model.parameters()])
# 精确地计算：1MB=1024KB=1048576字节
print('Number of parameter: % .7fM' % (total / 1e6))

```

```
Number of parameter: 0.4310800M
```

3.7 模型压缩并测试

使用pytorch自带的训练后动态量化，并输出量化后模型的性能指标

```
import torch.quantization
model_quantized = \
torch.quantization.quantize_dynamic(model_for_prune, {torch.nn.Linear}, \
dtype=torch.qint8)
```

压缩后测试结果：

Test set: Average loss: 0.0474, Accuracy: 9854/10000 (99%)

Number of parameter: 0.0255700M

 model2.pth	2024/11/...	PTH 文件	503 KB
 model1.pth	2024/11/...	PTH 文件	1,688 KB

4. 下载HuggingFace大模型，本地推理

4.1 transformers库安装

```
pip install transformers
```

4.2 加载BERT权重，使用BertModel生成最后一层隐藏层特征

隐藏层特征可输入到后续模块进行下游任务的微调

```
from transformers import AutoTokenizer, BertModel
inputs = tokenizer("I love large language model", return_tensors="pt")
outputs = model(**inputs)
last_hidden_states = outputs.last_hidden_state
print(last_hidden_states)

'''
```

```

output:
  tensor([[[[-0.1144, 0.1937, 0.1250, ..., -0.3827, 0.2107, 0.5407],
            [ 0.5308, 0.3207, 0.3665, ..., -0.0036, 0.7579, 0.0388],
            [-0.4877, 0.8849, 0.4256, ..., -0.6976, 0.4458, 0.1231],
            ...,
            [-0.7003, -0.1815, 0.3297, ..., -0.4838, 0.0680, 0.8901],
            [-1.0355, -0.2567, -0.0317, ..., 0.3197, 0.3999, 0.1795],
            [ 0.6080, 0.2610, -0.3131, ..., 0.0311, -0.6283, -0.1994]]],
        grad_fn=<NativeLayerNormBackward0>)
'''

```

4.3 进行掩码预测和下一句预测

```

from transformers import BertForMaskedLM
model = BertForMaskedLM.from_pretrained(checkpoint)
inputs = tokenizer("The capital of China is [MASK].", return_tensors="pt")
with torch.no_grad():
    logits = model(**inputs).logits
mask_token_index = (inputs.input_ids == tokenizer.mask_token_id)\
[0].nonzero(as_tuple=True)[0]

print(mask_token_index)
predicted_token_id = logits[0, mask_token_index].argmax(axis=-1)
tokenizer.decode(predicted_token_id)

'''
output:
  tensor([6])
  'beijing'
'''

```

```

from transformers import BertForNextSentencePrediction
from torch.nn import functional as F
model = BertForNextSentencePrediction.from_pretrained(checkpoint)

prompt = "i am from China"
next_sentence = "Attention is all you need"

```

```
encoding = tokenizer(prompt, next_sentence, return_tensors="pt")
outputs = model(**encoding, labels=torch.LongTensor([1]))
softmax = F.softmax(outputs.logits, dim=1)
print(softmax)
```

```
'''
```

```
output:
```

```
tensor([[0.0461, 0.9539]], grad_fn=<SoftmaxBackward0>)
```

```
'''
```

```
prompt = "Using a Transformer network is simple"
```

```
next_sentence = "Attention is all you need"
```

```
encoding = tokenizer(prompt, next_sentence, return_tensors="pt")
```

```
outputs = model(**encoding, labels=torch.LongTensor([1]))
```

```
softmax = F.softmax(outputs.logits, dim=1)
```

```
print(softmax)
```

```
'''
```

```
output:
```

```
tensor([[0.9980, 0.0020]], grad_fn=<SoftmaxBackward0>)
```

```
'''
```